Object Lens

System design and software quality

Private technical companion
Edition 2.3

Object Lens is a small local browser application for exploring object detection in a photograph. The user can choose a photo, inspect the model's predictions, adjust the minimum displayed score and save an annotated image. A sample-photo action provides a quick way to try the same workflow.

Source: docs/DESIGN.md

This companion explains what each part does, why it exists and how the application is checked. It is written as a learning reference for a validation engineer moving into software automation. The explanations follow one image through the application and one requirement through its automated test.

This document covers the application and the evidence for a bounded local release: the question it answers, stage-specific recovery, a loading deadline that tolerates slow but steady connections, visible-output assertions and box-position regression. The tested source is identified by file hashes. No public website deployment is included. Passing these checks does not establish general object-recognition accuracy.

Document control	Value
Prepared for	Tan Le Jie
Application source	be34c5e plus source hashes in the evidence record
Evidence version	docs/evidence/release-verification.json
Scope	Local Windows and Chrome learning and demonstration workflow

Contents

Scope and user workflow **4**
The question 4
Non-goals and rabbit holes 5
Objectives and completion 6
Technology and runtime architecture **6**
Server and worker boundaries 9
Configuration and loading feedback 9
Design decisions 10
Image processing and result behavior **10**
From file bytes to predictions 10
Current image ownership 12
Bounded recovery 12
Reject invalid model output 12
Filtering, samples and PNG output **13**
Minimum model score 13
Try sample 13
Save result 13
Testing with Python, pytest and Selenium **15**
The two commands 15
Controlled checks and actual inference 15
A worked requirement 16
Requirements and test design **17**
Ranked risks 20
Test mix 21
Choosing useful cases 21
Visible results and box regression 22
Failure investigation **22**
A real narrow viewport defect 23
A deliberate stale-result fault 24
A prediction mistake is a different kind of finding 24
An unexpected file in a download check 24
A slow connection that could never finish 25
Cache write, missing Content-Length and Clear during analysis 25
Stage-specific recovery wording 26
Verification and release assessment **26**
Earlier recorded runs 27
Core rules broken on purpose 29
Release decision 30
Running and maintaining the project **30**
Use the application 30
Run checks 30
Make a bounded change 31
AI assistance and honest demonstration 31
Interview story (draft) 31
Terms and source references **32**

Reading this companion

Start with the question and scope, then architecture, image processing and the separate testing path. Verification records the result and its limits. Figures are explanatory diagrams; screenshots are actual captures. Controlled failures, deliberate source faults and real model observations are identified separately.

Scope and user workflow

The intended user is a developer, quality engineer or learner who wants to try a pretrained detector and understand its predictions. Object Lens focuses on one photo at a time, one model and Chrome on Windows. Its small scope keeps the application and its automation understandable.

The question

When someone tries a browser object detector on their own photos, switching images quickly, on a slow connection, or when the model returns something broken, can the page show results that belong to a different photo, misplace boxes, or get stuck with no way to recover?

Source: docs/DESIGN.md

Measure: a controlled test for each of those failures, every core rule the answer depends on caught when broken on purpose, and a model download throttled to 2.5 Mbps compared with a fixed loading deadline and with stall-based loading.

Result: each failure has a controlled test, the suite has 102 tests, and every core rule made at least one test fail when it was broken on purpose (see Core rules broken on purpose in Verification and release assessment). At 2.5 Mbps, a fixed 120-second deadline failed at 121 s with 86 percent of the model received; the stall-based loading reached ready at 140.6 s.

Source: docs/evidence/release-verification.json; docs/evidence/release-sensitivity.json; docs/evidence/branch-review-slow-transfer.json

Scope the answer holds for: Windows with Chrome, one image at a time, the pinned model (Transformers.js 2.17.2 with a quantized DETR model), synthetic test images and small public photos. Failure cases use a controlled test worker; four tests run the real model. The answer says nothing about how accurate the model is.

The source repository is private. A static public package is prepared locally from an explicit allowlist, but it is not deployed and there is no verified public URL. A local server, source publication and deployment are separate operations.

The main workflow is to choose an image or try the sample, wait for inference, inspect the boxes and matching result rows, adjust the minimum score and save the visible annotations as PNG. Clear removes the current image and its results. Replacing a photo starts a new image workflow.

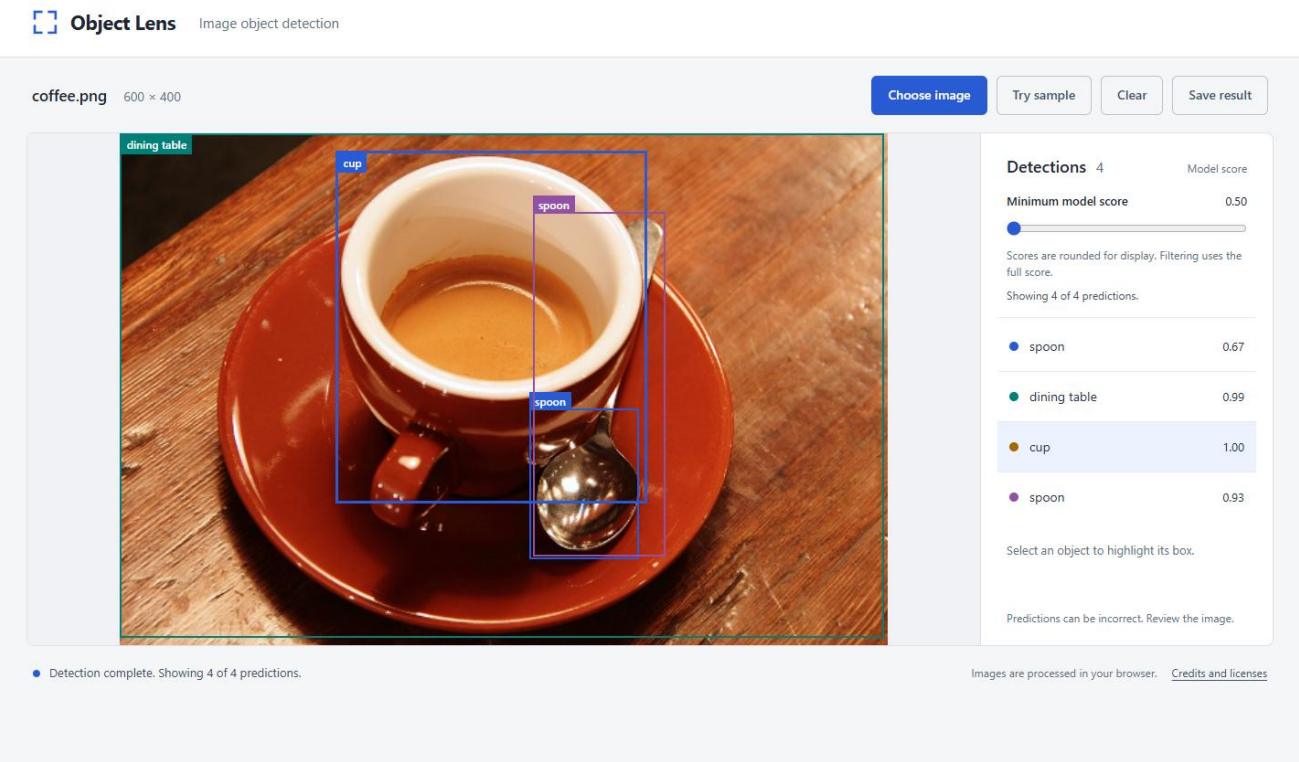


Figure 1 Application after real inference on the public coffee sample. The selected prediction links its visible row to its box; credits remain available.

Table 1 Product boundaries

Area	Included	Limit
Input	One decodable JPEG or PNG	At most 10 MiB and 16,000,000 pixels
Detection	One pretrained DETR model in the browser	Model classes and recognition limitations remain those of the chosen model
Interaction	Prediction selection, score filter, sample action, replace and Clear	No saved image history or batch workflow
Output	PNG with the current visible boxes, labels and scores	No report strip, cloud upload or accuracy certification
Execution	Local Windows application and headless Chrome tests	No hosted deployment, iOS or Safari qualification

Non-goals and rabbit holes

Each non-goal has a reason. The application also has no camera, video, batch processing, model picker or GPU tuning; these follow from the one-image, one-model scope.

Source: docs/DESIGN.md

Table 2 Non-goals

Non-goal	Reason
Model training or accuracy evaluation	The tests check how the app handles model output, not whether the model is accurate.
Accounts, upload records or image history	One image at a time, and photos never leave the browser, so nothing needs storing.
Hosted inference API or cloud deployment	The model runs locally in the user's browser.
Other browsers and mobile	Verified only on Windows with Chrome; the scope is kept small enough to test thoroughly.
Resumable model download	Try again restarts from zero; resuming would require changing the underlying library.

A rabbit hole is a piece of work that could absorb the project without answering the question. Each one below was removed by a decision recorded under Design decisions in the architecture chapter.

Table 3 Rabbit holes

Rabbit hole	Decision that removed it
Upgrading to Transformers.js v3	Stay pinned at 2.17.2 (D1).
Hand-made ground-truth boxes for the real-model checks	Compare with the pinned model's own recorded output and call it regression, not accuracy (D2).
Making a slow-network download reliable	A stall watchdog that asks "is it still moving?" instead of a fixed deadline, with no resume (D3).

Objectives and completion

The first objective is a usable path from an image to inspectable predictions. The second is to translate observable requirements into acceptance criteria and useful automated checks. The third is to retain enough evidence to investigate failures and make a limited, reasoned release decision.

Completion requires the features in docs/DESIGN.md to work together, relevant tests and UI checks to pass, an actual PNG export to match the visible predictions, and this document to describe the tested source. Source publication and online deployment are separate decisions.

Back to contents

Technology and runtime architecture

The web application and its automated tests are two different programs. Opening the application lets a person choose and inspect photos. Running pytest asks Python to execute checks against isolated application instances. The detector belongs to the application path; pytest and Selenium belong to the checking path.

Table 4 Technology responsibilities

Component	What it does	Why it is used here
HTML and CSS	Define the page, controls and presentation	A single page can use native browser features without a frontend framework
JavaScript	Validates input and coordinates image, prediction, filter and export state	The interface and model runtime execute in the browser
Web Worker	Runs model loading and inference away from the page's main thread	The page can respond to controls while model work is running
Transformers.js	Loads the selected model and calls its object-detection pipeline	Supplies a mature pretrained-model interface
ONNX Runtime Web	Executes the prepared model through CPU WebAssembly	Provides local inference in the target browser
Python HTTP server	Serves the application and prepared assets on 127.0.0.1	A small standard-library server is sufficient for local use
Static website package	Contains the same app, normal-mode configuration, model assets, sample and credits	Allows ordinary static serving without the custom Python routes
Python and pytest	Define checks, prepare fixtures and report assertions	Keep the core automation in one language with a mature test runner
Selenium	Operates an isolated Chrome session and reads its page	Exercises the user-visible browser behavior



Figure 2 Runtime responsibilities. The local server provides files; the browser performs inference and draws PNG output. The separate automated-testing workflow appears in Figure 5. This figure illustrates local Python serving; the static package supplies the same browser files through an ordinary static server.

The prepared application uses local runtime and model files. The worker sets remote-model access off and requests the pinned local model. An initial setup still needs downloads for dependencies, model assets, public samples and potentially the matching Chrome driver. Local inference therefore means that the prepared inference path does not call a hosted prediction service.

The application uses Transformers.js 2.17.2 and ONNX Runtime Web 1.14.0 with a quantized Xenova/detr-resnet-50 model. Exact sources, model revision, file sizes and SHA-256 hashes are recorded in asset-lock.json. The version pins make the tested setup reproducible; they do not mean that these are the newest releases.

Server and worker boundaries

The Python server binds to 127.0.0.1, which addresses this computer. It is not an online demo service. The browser worker owns model loading and inference. The page owns the chosen image, the current request identity and everything displayed to the user.

Running inference in a worker keeps model execution out of the main page thread, but it does not make the computation free or provide a speed guarantee. This version uses CPU/WASM with one thread. It makes no GPU, NPU or semiconductor validation claim.

Configuration and loading feedback

The page first loads runtime configuration under a 15-second deadline covering response headers and the JSON body. The testMode value must be a boolean before a worker can start. A failed request exposes retry and retains a valid selected image. A new valid selection can also retry configuration.

Source: dist/app.js; tests/test_startup.py

Model feedback separates transfer progress from runtime initialization. A percentage is shown only after an event reports fewer bytes than its total. Without a Content-Length header, Transformers.js 2.17.2 reports the bytes received so far as the total, so the page then shows megabytes received with an indeterminate bar. A transfer at 100 percent is not readiness; only the worker ready event establishes that. Worker callbacks must still belong to the current worker instance. Background startup cannot erase an input error or say the page is idle while a file is still being read.

Source: dist/app.js; dist/worker.js; tests/test_loading.py

The static package must be served at the site root because worker asset paths begin with /assets. Browser caching may retain model/runtime downloads; the app keeps selected photo data and current predictions in its page/worker and provides no persistent photo-history feature. Clear resets current image state, not cached model assets.

Source: dist/worker.js; dist/app.js; docs/PUBLIC_DEMO.md

Design decisions

Three decisions shape the application and its tests. Each records the context, the option chosen, the alternatives weighed and the cost accepted.

Source: docs/DESIGN.md

Table 5 Design decisions

Decision	Context and choice	Alternatives weighed	Cost accepted
D1 Run a pinned model in the browser	Detection on the user's own photos without accounts or a server that stores images. Transformers.js 2.17.2 runs a local quantized DETR model in a Web Worker, pinned to that version.	A hosted inference API, which sends photos off the device; upgrading to Transformers.js v3, which changes the API used by the original example.	CPU-only analysis of about 8 to 14 seconds per image, a 43 MB first download and an older library.
D2 Controlled worker for most tests, real model for a few	The important risks are timing, stalls, crashes and malformed output, which the real model cannot produce on demand. 98 tests run without the real model; 4 run it.	Testing everything against the real model, which is slow and cannot create most failure cases reproducibly.	Most tests never run the model, so model-side problems rely on the 4 real-model tests and manual observation.
D3 Judge model loading by stalls, not total time	A fixed total deadline failed a steady 2.5 Mbps download at 86 percent. Loading fails only after 45 seconds without progress; initialization after the transfer has its own 120-second deadline.	A longer fixed deadline such as 300 seconds; a resumable download, which needs changes to the underlying library.	Try again restarts the download from zero, and a real disconnect takes 45 seconds to report.

Back to contents

Image processing and result behavior

From file bytes to predictions

When a person chooses a file, the application checks the allowed format, file size, successful decoding and image dimensions. A filename extension alone is insufficient: a file called photo.png can contain unreadable or unsupported bytes. Validation prevents unsuitable input from reaching the expensive model call.

The real worker passes the image to the object-detection pipeline. The model uses learned parameters from its earlier training to compute predictions from the image pixels. The application does not select a predefined answer from the filename, and uploading an image does not train or update the model.

Excerpt 1 Real inference call in the worker

```
const predictions = await detector(data.image, { threshold: 0.5, percentage: true });
```

Source: dist/worker.js: infer

Table 6 One prediction

Field	Meaning	How the interface uses it
label	A supported class name predicted by the model	Text in the result row and box label
score	The model's score for that prediction	Displayed beside the label and compared with the display minimum
box	Normalized xmin, ymin, xmax and ymax coordinates	Position and size of the box relative to the decoded image
Stable prediction identity	An application identity retained for the current result set	Keeps selection and color associated with the same prediction when filtering

A score is not a measured accuracy percentage. Two predictions can describe the same visible object, and a high-scoring prediction can still be incorrect. The interface therefore counts predictions rather than claiming a verified count of physical objects.

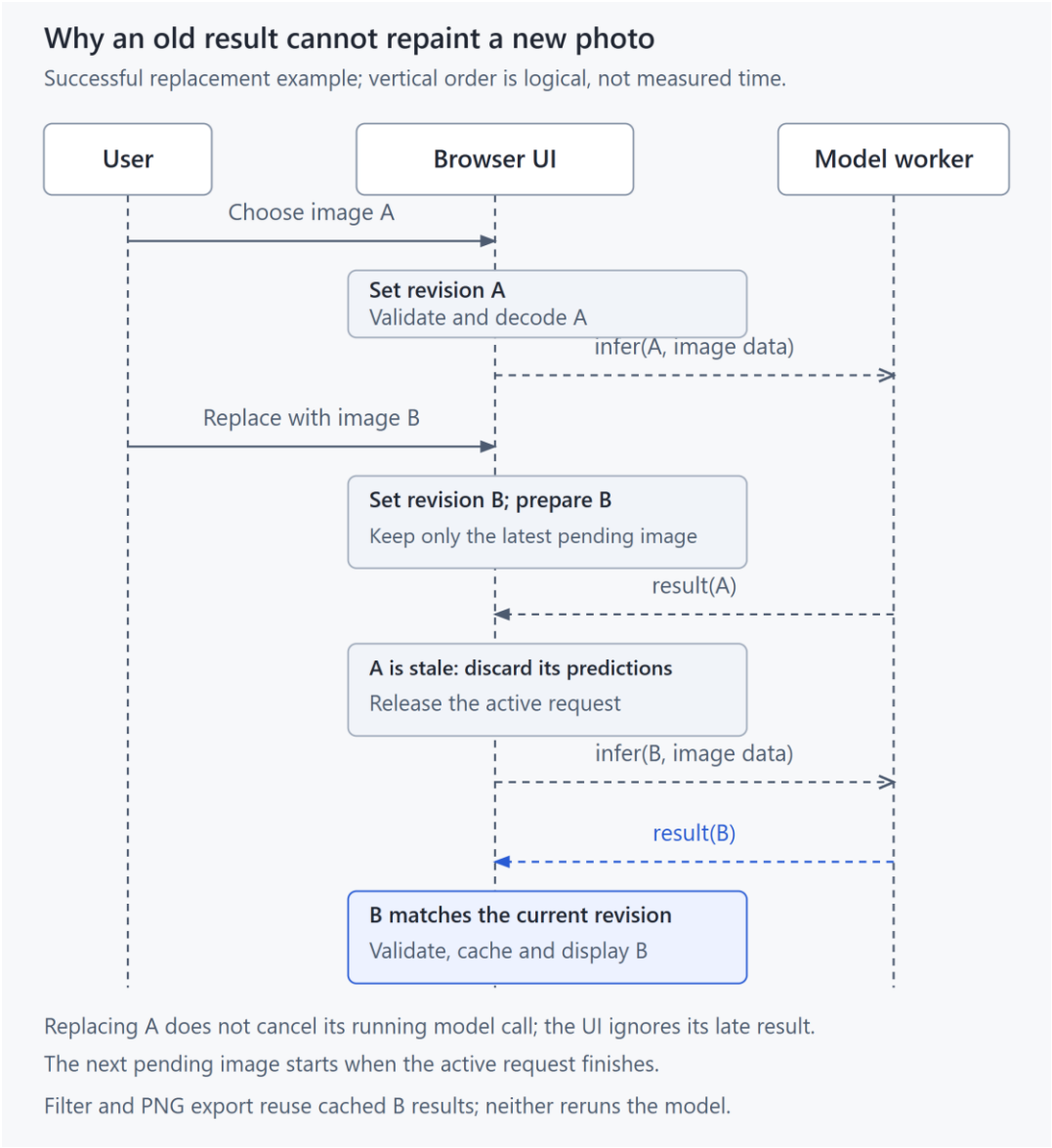


Figure 3 Image A finishes after the user selects image B. The application discards A's stale reply, then processes the latest pending image. Filtering and PNG export use retained results after inference.

Current image ownership

Every new image workflow has a revision identity. A result is accepted only when its request belongs to the current image. If an earlier image finishes after replacement or Clear, its reply cannot repaint the new page. This is an important distinction: discarding an old reply does not promise that the CPU work was interrupted.

The worker handles one active request, while the page retains at most the latest pending image. This prevents rapid replacements from turning into an unbounded list of obsolete requests.

Bounded recovery

Model transfer fails only after 45 seconds without a loading event, so a slow but steady connection can finish the 43 MB download. Once every byte of a known-size model transfer has arrived, runtime initialization has a fixed 120-second deadline that also covers the library's silent Cache Storage write, and inference has 90 seconds. The failed worker is terminated. A stalled transfer asks the user to check the connection, an initialization timeout says the model took too long to start, and an inference timeout describes image analysis. If Clear removed an image whose analysis then times out, the detector restarts quietly with no failure shown on the empty workspace. Otherwise Clear removes the photo and replaces the earlier explanation with a generic detector-unavailable message offering restart. It does not claim readiness or reclassify the prior failure as startup failure. Retry or a new valid image recreates the worker.

Source: dist/app.js; tests/test_recovery.py

Normal controlled-worker loading retains the production deadlines. Dedicated loading tests accelerate only the 45-second stall and 120-second initialization timers in their isolated browser, and check a stalled transfer, a slow steady transfer that must not time out, a completed transfer followed by silence, and an initialization hang. Controlled inference uses a four-second deadline. A simulated failure proves that recovery works under the chosen condition, not that the real model was observed hanging.

An empty prediction set, an invalid image and an inference failure have different meanings. Empty results mean the detector returned no predictions under its configured output threshold. Invalid input failed validation. An inference failure means the requested processing did not complete successfully. The page should communicate the correct state and offer the applicable recovery.

Reject invalid model output

The page validates the returned array and each prediction before displaying results. Labels must be nonblank strings, scores must be finite numeric values from 0 to 1, and the four coordinates must be finite numbers. Coordinates are clamped to the image bounds only if the resulting rectangle has positive area. Missing fields, invalid scores and reversed, collapsed or wholly out-of-bounds rectangles are rejected.

Source: dist/app.js; tests/test_recovery.py

A malformed prediction invalidates the response as a whole, including a batch with an otherwise valid entry. The page reports the problem, leaves no misleading boxes or result rows, and allows a later valid response to recover. The controlled fixtures intentionally create these failures; the real-model observations did not produce them.

Back to contents

Filtering, samples and PNG output

Minimum model score

The display filter ranges from 0.50 to 1.00 in steps of 0.01. It starts at 0.50 for a newly chosen or sample image and resets on Clear. The filter compares the unrounded model score with the selected minimum and includes a prediction when its score is greater than or equal to that value.

The real detector is still requested with its existing 0.5 output threshold. Raising the display minimum hides predictions already returned by that call; lowering it restores retained predictions down to the available 0.50 floor. The filter cannot recover predictions the detector did not return, and it does not change the model's parameters.

In the recorded coffee observation (model-observations.json), one spoon prediction scored about 0.67 and another about 0.93. A display minimum of 0.80 would hide the first and retain the second. This is a concrete filtering example, not evidence that the retained prediction is necessarily correct.

The boxes, list and visible count update from the same filtered set. Predictions retain their original identities and colors. If the selected prediction remains visible, selection is preserved. If it is hidden, the first remaining prediction becomes selected; if none remains, selection is cleared.

Source: dist/app.js; tests/test_filter.py

When the model returned predictions but the filter hides all of them, the page says that no predictions meet the minimum score and invites the user to lower it. The image stays visible and the slider stays enabled. If the detector returned an empty set, the page uses a different empty message. Displayed scores are rounded to two decimals; comparisons use the original score.

Source: dist/app.js; tests/test_filter.py

Try sample

Try sample loads the existing public coffee photo through the ordinary input and real-inference flow. Its purpose is to make the application easy to try without browsing through folders. The photo is a genuine input, not a shortcut to a fixed prediction list.

The sample file and its attribution already belong to the prepared local assets. A missing or unreadable sample produces an actionable error and a new Try sample attempt can recover. A 15-second deadline aborts a fetch that does not finish.

Source: dist/app.js; tests/test_sample.py

Sample loading reserves a new image revision before starting its fetch. Clear or a newer image selection aborts that request and retires its identity. A late response cannot become the current image. Controlled delayed-response tests exercise both Clear and replacement; this is verified prevention of a design risk, not a claim that a production incident occurred.

Source: dist/app.js; tests/test_sample.py

Save result

Save result creates a PNG of the decoded, correctly oriented image at its original pixel resolution, with only the currently visible prediction boxes, labels and model scores. It includes no summary strip. The source image file is not overwritten.

The on-screen boxes are HTML elements over the image. PNG export draws the same visible prediction data onto a canvas in the page's main thread. Coordinates are scaled to the decoded image

dimensions, and text labels include the class and rounded model score. Label backgrounds and text are kept within the canvas. The saved PNG is therefore checked separately from the screen.

Source: dist/png-export.js; tests/test_export.py

Export captures one image, one visible prediction set and the selected minimum. Clear, replacement or a filter change invalidates an export still being encoded, so it cannot initiate a stale download. Save is disabled when no predictions are visible and while an export is being prepared. An encoder failure or the 15-second encoding deadline leaves the displayed result intact and makes Save available for another attempt.

Source: dist/app.js; dist/png-export.js; tests/test_export.py

The application reports PNG download started after handing the generated file to the browser. It does not claim that the operating system has confirmed a completed save. The filename includes the source image stem and the display minimum, for example coffee-object-lens-min-0.80.png.

Source: dist/app.js; dist/png-export.js

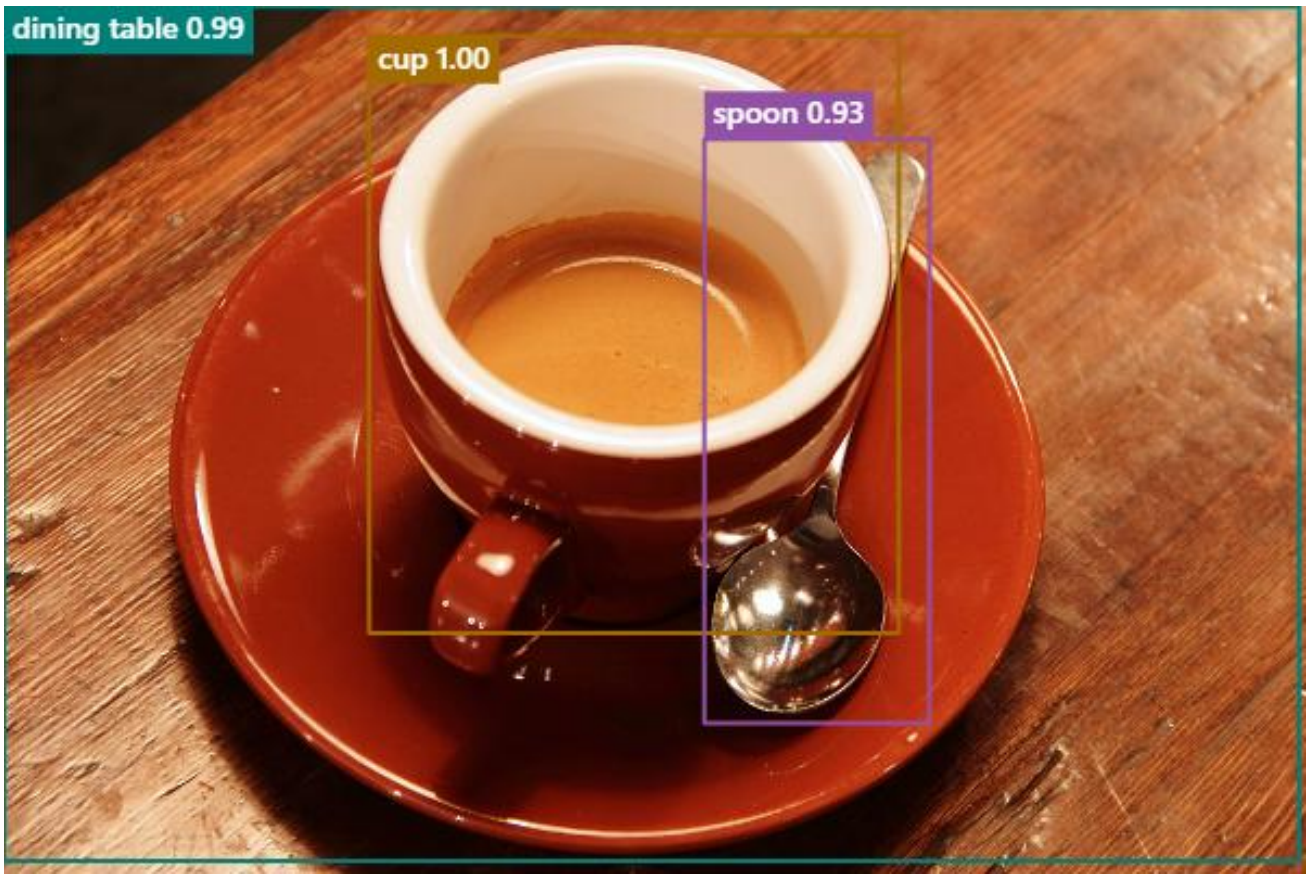


Figure 4 Actual PNG saved from the coffee sample at minimum score 0.80. It contains the photo and three visible prediction annotations, without application controls.

The saved image contains model predictions. A recipient should still review the photo and interpret the labels as predictions rather than verified facts.

Back to contents

Testing with Python, pytest and Selenium

The two commands

Table 7 Application and test commands

Command from the project folder	What it does	Why to use it
.\.venv\Scripts\python.exe serve.py	Starts the local application server	Use or demonstrate the application manually
.\.venv\Scripts\python.exe -m pytest	Runs the written automated tests	Check whether application behavior matches the expected rules

The server command stays running until it is stopped. The pytest command finishes after the test functions and their cleanup. Tests start their own temporary servers and headless Chrome sessions, so the manually started app is not a prerequisite.

Python supplies the language in which the tests are written. pytest discovers test files, arranges fixtures, executes test functions and reports outcomes. Selenium follows the actions described by a test: selecting a file, clicking a control and reading the displayed page. Assertions compare the observed values with the expected rule.

Controlled checks and actual inference

The controlled test worker deliberately supplies known predictions, delays and failures. This lets a test reliably exercise an empty set, a timeout or a stale response. It does not establish how accurately the real model recognizes objects. Normal application mode uses the real worker; controlled mode must be enabled explicitly and is visibly labelled.

Tests marked inference run the actual prepared model. Three photo cases assert person, cup and cat text/scores, compare the selected coordinates with earlier pinned-model outputs and measure actual rendered geometry. A fourth case serves the generated package through an ordinary static server and exercises actual inference, filtering and PNG download. These are limited integration/regression checks, not an accuracy benchmark.

Run a focused group

```
# Non-model application, server, package and harness checks
.\.venv\Scripts\python.exe -m pytest -m "not inference"

# Actual-model checks
.\.venv\Scripts\python.exe -m pytest -m inference
```

In the second command, Python interprets the first -m as run this installed module. pytest interprets the later -m as select tests with this marker. The identical spelling belongs to two different command-line interfaces.

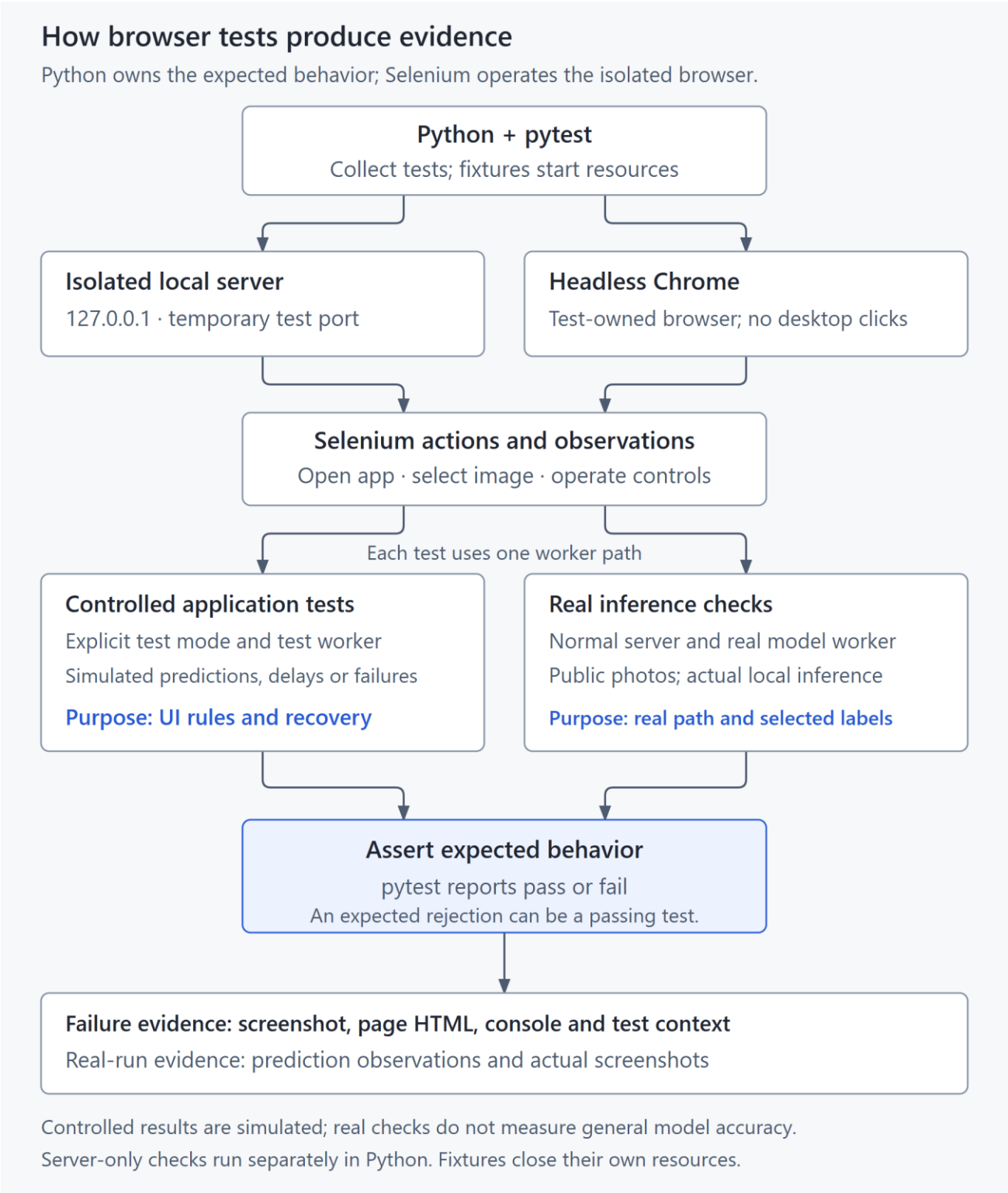


Figure 5 pytest runs written checks through Selenium. Controlled worker responses make interface scenarios repeatable; separately marked checks exercise the actual local model.

A worked requirement

The maximum file size is inclusive: a valid file exactly 10 MiB must be accepted, while a file one byte larger must be rejected. This is a boundary test because defects often occur at the point where accepted input becomes rejected input. A test far below the limit would not establish the inclusive rule.

Excerpt 2 Inclusive byte-size boundary and useful error content

```
def test_file_size_boundary(app, make_image, size, expected_state):
    upload(app, make_image("size-boundary.png", total_bytes=size))
    wait_state(app, expected_state, timeout=20)
    if expected_state == "error":
        assert_actionable_error(app, "10 mib", "smaller", "jpeg", "png")
        assert result_labels(app) == []
    else:
        assert result_labels(app) == ["laptop", "cup"]
```

Source: tests/test_ui.py: test_file_size_boundary

In tests/test_ui.py, the parameterization above this function runs the same test at two sizes: exactly 10 MiB and one byte above it. Read the function in four parts: prepare a valid image with the chosen byte count; submit it through Selenium; wait for an observable state; assert the expected rejection or controlled prediction list. The image must remain decodable so the case reaches the intended size rule. The laptop/cup labels are deliberate test-worker responses, not recognition results.

A negative test can pass when the application displays an error. For an oversized file, rejection with a useful message is the required behavior. pytest reports failure when the observed behavior differs from that requirement; it does not independently decide what the product ought to do.

Back to contents

Requirements and test design

Acceptance criteria describe externally observable behavior. The test plan chooses inputs and actions that could reveal a violation. Mapping a test to a requirement makes a failure interpretable and exposes requirements that have only manual or limited evidence.

Table 8 Acceptance examples and named tests

Criterion	Given / When / Then	Named tests
AC1 Input limits	Given a valid PNG of exactly 10 MiB, when the user selects it, then it is analysed; given one byte more, then an error names the 10 MiB limit and no predictions appear. 4000 by 4000 pixels is analysed; 4001 by 4000 is rejected.	tests/test_ui.py:: test_file_size_boundary tests/test_ui.py:: test_decoded_pixel_boundary tests/test_ui.py:: test_invalid_input_is_rejected_and_valid_input_recovers tests/test_ui.py:: test_truncated_png_body_is_rejected_and_recovers
AC2 Results match the current image	Given a photo for which the model returns laptop (0.97) and cup (0.91), when analysis completes, then the list shows exactly "laptop 0.97" and "cup 0.91", each with one labelled box at the returned coordinates within 2 CSS pixels.	tests/test_ui.py:: test_predictions_and_boxes_match_the_current_image tests/test_ui.py:: test_jpeg_exif_orientation_preserves_display_and_box_alignment tests/test_inference.py:: test_real_model_detects_reviewed_object

Criterion	Given / When / Then	Named tests
AC3 Selection and Clear	Given laptop and cup results, when the user selects cup, then only the cup box is highlighted; when the user activates Clear with the keyboard, then the image, boxes and list are removed.	tests/test_ui.py:: test_selecting_a_result_highlights_its_box tests/test_ui.py:: test_clear_is_reachable_and_usable_with_keyboard
AC4 An old result cannot repaint	Given a dog photo still being analysed, when the user selects a car photo, then "dog" never appears; when the user presses Clear instead, then the workspace stays empty.	tests/test_ui.py:: test_replacing_a_processing_image_discards_the_old_result tests/test_ui.py:: test_clearing_a_processing_image_prevents_late_repaint
AC5 Failures are visible and recoverable	Given an analysis that fails, returns malformed output, never answers or crashes the worker, when it ends, then the page says what failed with Try again and shows no partial predictions, and the next valid image or Try again recovers. An empty result is not an error.	tests/test_ui.py:: test_empty_predictions_are_not_an_error tests/test_ui.py:: test_nonresponding_worker_times_out_and_retries_the_latest_image tests/test_recovery.py:: test_malformed_result_is_rejected_without_partial_detections_and_recovers tests/test_recovery.py:: test_inference_timeout_clear_and_retry_describe_detector_availability
AC6 Keyboard, announcements and narrow screens	Given a completed result, when the user tabs through the page, then Clear, Try sample and the minimum-score control are reachable and usable; at 320 or 390 CSS pixels the page does not scroll sideways.	tests/test_ui.py:: test_clear_is_reachable_and_usable_with_keyboard tests/test_ui.py:: test_status_and_error_announcement_configuration_follows_visible_states tests/test_ui.py:: test_narrow_layout_preserves_results_and_does_not_overflow tests/test_sample.py:: test_try_sample_is_keyboard_reachable_and_activates_with_enter tests/test_filter.py:: test_minimum_score_has_accessible_label_and_keyboard_controls
AC7 Score filter	Given spoon (0.7999), cup (0.80) and bottle (0.95), when the minimum is set to 0.80, then cup and bottle remain in the list and the boxes; when it returns to 0.50, then all three return without a second inference request.	tests/test_filter.py:: test_exact_minimum_filters_underlying_scores_and_restores_without_inference tests/test_filter.py:: test_filter_preserves_result_identity_color_and_surviving_selection

Criterion	Given / When / Then	Named tests
AC8 Try sample	Given a held sample response, when the user presses Clear or selects car.png before it arrives, then after the response is released the workspace stays empty or keeps showing car.	tests/test_sample.py:: test_late_sample_response_cannot_override_a_later_user_action tests/test_sample.py:: test_sample_fetch_or_decode_failure_can_be_retried
AC9 PNG export	Given a 640 by 480 photo filtered at 0.80, when the user presses Save result, then the PNG is 640 by 480 with the cup box and no pixels of the hidden spoon box; when Clear, a new image or a filter change happens during encoding, then nothing downloads.	tests/test_export.py:: test_downloaded_png_contains_only_filtered_boxes_at_original_dimensions tests/test_export.py:: test_changed_result_cancels_a_png_still_being_encoded
AC10 Startup and model loading	Given a transfer that keeps reporting progress for longer than several stall windows, when it finishes, then it is not timed out; given a transfer that stops reporting progress, then the page reports the stall 45 seconds after the last event and asks the user to check the connection.	tests/test_recovery.py:: test_slow_steady_model_transfer_is_not_timed_out tests/test_recovery.py:: test_stalled_model_transfer_times_out_after_its_last_progress tests/test_recovery.py:: test_model_initialization_has_its_own_deadline
AC11 Public package	Given the built package on an ordinary static server, when the user tries the sample, filters at 0.80 and saves, then the real model returns a cup and the PNG is 600 by 400. The ZIP contains exactly the manifest files.	tests/test_public_package.py:: test_public_zip_exactly_matches_manifest_and_excludes_private_material tests/test_public_package.py:: test_packaged_site_runs_real_sample_filter_and_export_on_an_ordinary_static_server tests/test_server.py:: test_controlled_worker_requires_explicit_test_mode

Source: docs/DESIGN.md

Each row is one concrete example of its criterion, with the tests that check it. docs/DESIGN.md gives every example in full, the complete test list for each criterion and the part of the question it answers; AC3, AC6, AC7 and AC11 are supporting criteria that keep the app usable. The three tests in tests/test_harness_cleanup.py check the test harness itself and map to no criterion.

Source: docs/DESIGN.md

The named tests identify assertions in the documented source. The execution record in Verification and release assessment identifies which checks actually ran. Controlled fixture passes establish the specified application behavior, while actual-model runs supply a different and limited form of evidence.

Ranked risks

The failures most likely to hurt the intended user are ranked below, most important first, with the tests that cover them. The last three rows are known risks that are not tested, with the reason.

Source: docs/DESIGN.md

Table 9 Ranked risks and covering tests

Rank	Risk	Likelihood and impact	Covering test
1	An old photo's result is shown on a new photo	Likely: analysis takes 8 to 14 seconds. High: the user trusts labels that belong to another photo.	tests/test_ui.py:: test_replacing_a_processing_image_discards_the_old_result tests/test_ui.py:: test_clearing_a_processing_image_prevents_late_repaint
2	The model never finishes loading on a slow connection	Fairly likely: the model is a 43 MB download. High: the app is unusable.	tests/test_recovery.py:: test_slow_steady_model_transfer_is_not_timed_out tests/test_recovery.py:: test_stalled_model_transfer_times_out_after_its_last_progress the 2.5 Mbps probe in branch-review-slow-transfer.json
3	The page hangs or the detector crashes with no way to recover	Moderate. High: the user has to reload and may lose the photo.	tests/test_recovery.py:: test_inference_timeout_clear_and_retry_describe_detector_availability tests/test_ui.py:: test_nonresponding_worker_times_out_and_retries_the_latest_image
4	Boxes are drawn in the wrong place	Low: the model is pinned. Medium to high: a correct label points at the wrong object.	tests/test_inference.py:: test_real_model_detects_reviewed_object tests/test_recovery.py:: test_partially_outside_box_is_clamped_to_the_rendered_image
5	Broken model output is shown as a result	Low. High: nonsense appears as a real detection.	tests/test_recovery.py:: test_malformed_result_is_rejected_without_partial_detections_and_recovers (16 cases)
6	A valid photo is rejected, or an oversized photo freezes the tab	Moderate. Medium.	tests/test_ui.py:: test_file_size_boundary tests/test_ui.py:: test_decoded_pixel_boundary
7	The exported PNG differs from what is on screen	Moderate. Medium: the saved file misrepresents the result.	tests/test_export.py:: test_downloaded_png_contains_only_filtered_boxes_at_original_dimensions tests/test_export.py:: test_changed_result_cancels_a_png_still_being_encoded

Rank	Risk	Likelihood and impact	Covering test
-	Repeated 45-second stalls on a poor connection	Not rated	Not tested, because resuming a download is a non-goal; each Try again restarts the download from zero.
-	Other browsers and mobile	Not rated	Not tested, because only Windows with Chrome is in scope.
-	Large real phone photos with EXIF rotation	Not rated	Not tested, because only synthetic images and small public photos are used; EXIF rotation is tested on a small synthetic JPEG.

Test mix

Each layer exists because of the risks it can reach. The counts are the cases collected in the recorded full suite.

Source: docs/DESIGN.md; docs/evidence/release-full-suite.xml

Table 10 Test mix with actual counts

Layer	Why this layer	Tests
Unit (browser-free Python)	The public package must not ship test mode, private files or changed assets, must take git provenance only from its own repository, and the harness must not leave browsers running; none needs a browser. 3 in test_harness_cleanup.py, 10 in test_public_package.py.	13
Integration	A normal server must never serve the controlled test worker, otherwise fabricated predictions could be presented as model results (test_server.py).	1
Browser end-to-end with the controlled worker	Risks 1, 2, 3, 5, 6 and 7 need exact timing, stalls, crashes and malformed output on demand. 25 in test_ui.py, 26 in test_recovery.py, 8 in test_export.py, 7 in test_filter.py, 7 in test_sample.py, 6 in test_startup.py, 5 in test_loading.py.	84
Real-model end-to-end	Risk 4 and the real inference path need the actual model; each run is slow, so there are few. 3 in test_inference.py, 1 in test_public_package.py.	4
Total		102

Choosing useful cases

The score-filter checks include 0.7999, exactly 0.80 and 0.95 against a minimum of 0.80. The first prediction is excluded even though its displayed rounded score is 0.80; the other two remain. A message counter verifies that filtering and restoring predictions do not trigger a second inference. Separate cases check stable identity, color, selection, an all-hidden state and new-image reset.

The PNG checks verify decoded dimensions, including a JPEG stored at 320 by 240 pixels that displays and exports at 240 by 320 after EXIF rotation. Known pixel locations check for a retained box, an excluded box and an unchanged background. Encoder-failure and delayed-callback cases test recovery and cancellation. The actual-model coffee export separately demonstrates the full integration.

Source: tests/test_export.py

Exploratory testing complements the scripted cases. A short charter is to change input while the app is busy, adjust the filter with the keyboard and inspect whether every visible result still belongs to the displayed image. Record the inputs and observations so an unexpected result can be reproduced.

A long repeated-use session checks a different risk from a single functional test. It can reveal observed state or recovery failures during that session, but it does not certify memory-leak freedom, production availability or concurrent-user scalability.

Status, input errors, loading feedback and export feedback have automated checks for useful visible content and the intended announcement configuration. The export-message element remains present as a polite live region even when empty. These checks inspect page content and DOM settings; they do not certify actual screen-reader speech or complete accessibility.

Source: dist/index.html; dist/style.css

Visible results and box regression

Reading only data attributes can miss an empty visible label. The checks inspect rendered names in result rows and corresponding box labels, and formatted scores in result rows. Deliberately blanking that text in a disposable copy must fail the assertions even if data-label and data-score remain correct.

Source: tests/test_ui.py; tests/test_inference.py; tools/check_review_sensitivity.py

The three reference boxes come from the earlier output of the same pinned model in model-observations.json. Each normalized edge may differ by 0.005, which is 0.5 percent of the image extent and about 1.5 to 3 source pixels for these samples. Rendered positions allow that scaled edge tolerance plus 2 CSS pixels for layout rounding/borders; widths and heights allow two edge tolerances plus 2 CSS pixels. Actual rendered position and size are measured relative to the displayed image. Deliberately displacing every box must fail even though its label and count remain unchanged.

Source: tests/test_inference.py; docs/evidence/model-observations.json

These comparisons answer whether a selected result or its rendering changed. Independent human box annotations would be needed to assess localization accuracy. Neither a good match nor a high score establishes that all predictions are correct.

Back to contents

Failure investigation

A useful failure report states the expected rule, the exact steps and input, the observed behavior and the evidence needed to reproduce it. Start with what happened before proposing a cause. A screenshot shows the visible symptom; browser logs, test assertions and request identities can explain the path that produced it.

Table 11 Evidence to retain

Evidence	Question it helps answer
Test name, assertion and command	Which expected rule disagreed with the observed state
Input and controlled scenario	Whether another run can repeat the same conditions
Screenshot and page HTML	What the page actually displayed at the failure
Browser console output	Whether runtime or worker errors accompanied the symptom
Source version and environment	Which implementation and browser/runtime were exercised
Rerun after the fix	Whether the proposed fix addressed the original reproduction

A real narrow viewport defect

A genuine layout defect occurred at a 320 CSS-pixel viewport. A vertical scrollbar left 305 pixels available to the document, but the body still had a 320-pixel minimum width. The page overflowed horizontally. The preserved failing test reports the comparison 320 <= 305 + 1 as false.

Table 12 Narrow viewport before and after the correction

Measurement	Before	After
Requested CSS viewport	320 px	320 px
Available document width	305 px	305 px
Document content width	320 px	305 px
Overflow beyond available width	15 px	0 px

The earlier check compared content width with window.innerWidth, which included space unavailable to the content when the scrollbar was present. It could therefore pass while the page still overflowed. The corrected check sets the CSS viewport explicitly and compares scrollWidth with documentElement.clientWidth, allowing one pixel for rounding. It also checks that each essential control fits within the available width.

Changing the body's min-width from 320px to 0 lets the responsive layout use the actual available width. A real-model capture of the corrected layout measured content and available widths of 305 and 305 at 320, and 375 and 375 at 390. The 768- and 1440-pixel layouts also fit their available widths. This finding combines an application fix with a stronger test measurement; it was not an injected source fault.

Source: docs/evidence/viewport-before-fix.xml; dist/style.css; tests/test_ui.py; docs/evidence/extension-demo.json

A deliberate stale-result fault

The sensitivity run disables the current-image revision guard in a disposable copy: the condition data.id === revision && currentImage?.id === revision becomes true. A delayed result for an earlier image can then appear on a replacement image, and tests/test_ui.py::test_replacing_a_processing_image_discards_the_old_result[slow-car.png] and tests/test_ui.py::test_clearing_a_processing_image_prevents_late_repaint fail. This demonstrates that the checks can expose that chosen fault. The fault is introduced deliberately in an isolated copy; it is not a naturally discovered production incident.

Source: docs/evidence/release-sensitivity.json

The investigation connects the symptom to a narrow mechanism: an old request returned after the displayed image changed, and the altered copy no longer rejected it. Restoring the guard addresses that mechanism. This is first-level application investigation, not a claim about a defect inside the pretrained model.

A prediction mistake is a different kind of finding

The real coffee run included duplicate spoon predictions, while another sample produced incorrect and duplicate labels. Those observations describe model behavior. If the application accurately renders the returned data, an incorrect model label is not by itself proof that the rendering code is broken.

The response should match the finding: fix a reproducible application defect, improve the wording if the UI overstates a prediction, or record a model limitation. Raising a score filter may hide some predictions but does not establish general accuracy improvement.

An unexpected file in a download check

One full-suite run (extension-first-full-suite.xml) recorded 47 passes and one failure. The failure occurred when a timeout test expected an empty download folder but found downloads.htm.crdownload. The retained investigation identified a non-PNG CRX3 container holding model data. The producing process or component was not established, so the finding does not prove that Object Lens generated that file.

Source: docs/evidence/extension-first-full-suite.xml; docs/evidence/extension-verification.json

The strict assertion was retained. The isolated test browser is configured to resolve only localhost and 127.0.0.1 because its application and model assets are served locally. This setting is confined to the test browser and does not alter the computer's network or security settings. A complete rerun with that setting passed all 48 cases.

Source: tests/conftest.py; docs/evidence/extension-full-suite.xml

A slow connection that could never finish

This defect is shown in the five parts used for every defect in docs/SOURCE_REVIEW.md: what failed, how it was detected, its impact, the cause in the system and the test that prevents it.

Source: README.md; docs/SOURCE_REVIEW.md

Table 13 The slow-connection defect in five parts

Part	Detail
What failed	With only the 43.1 MB model download throttled to 2.5 Mbps on loopback, the app showed "The detection model took too long to load" after 121 s, with 37.4 of 43.1 MB received (86 percent).
How it was detected	An independent review ran a paced loopback probe at 2.5 Mbps against the package (branch-review-slow-transfer.json).
Impact	Try again restarted the download from zero, so at this speed retrying could never succeed and the detector could not be used. A steady connection below about 2.9 Mbps could never load the model.
Cause in the system	Model loading had one fixed 120 s deadline, counted from worker start. It ignored download progress, so a slow but steady download was treated the same as a dead one.
Test that prevents it	Download stalls and slow but steady downloads: the download fails only after 45 s with no loading event, and each event restarts that window. tests/test_recovery.py:: test_slow_steady_model_transfer_is_not_timed_out tests/test_recovery.py:: test_stalled_model_transfer_times_out_after_its_last_progress Start-up after the last byte has its own fixed 120 s deadline, which later events cannot extend. tests/test_recovery.py:: test_model_initialization_has_its_own_deadline tests/test_recovery.py:: test_completed_transfer_starts_initialization_before_the_done_event

Source: docs/evidence/branch-review-slow-transfer.json; dist/app.js; tests/test_recovery.py

The deadline was protecting against a dead connection, not a slow one, so loading measures a stall rather than total time. Under the same 2.5 Mbps probe, the package received every byte and reached ready at 140.6 s. Each condition ran once, on loopback; that is not a measurement of Internet speed. Three of the deliberate faults in release-sensitivity.json guard this behavior: a fixed load deadline, initialization extended by later events, and a completed transfer left under the stall window. This was an observed defect, not an injected fault.

Source: docs/evidence/branch-review-slow-transfer.json; docs/evidence/release-sensitivity.json

Cache write, missing Content-Length and Clear during analysis

Three related behaviours each have a test. The library writes the downloaded model to Cache Storage before reporting done, so that local write ran under the connection-stall window; once every byte of a known-size transfer has arrived, the initialization deadline applies instead (tests/test_recovery.py::test_completed_transfer_starts_initialization_before_the_done_event). Without a Content-Length header the library reports the bytes received as the total, so the page showed 100 percent during the whole transfer; the page shows megabytes received with an

indeterminate bar (tests/test_loading.py::test_missing_content_length_shape_stays_indeterminate). After Clear, a hung analysis of the removed image produced an analysis failure on an empty workspace; the detector restarts quietly (tests/test_recovery.py::test_clear_during_a_hung_analysis_restarts_quietly).

Source: docs/SOURCE_REVIEW.md

The evidence checks also catch miscounted deliberate faults, a score check that tolerated every score forced to 1.0, startup tests blind to a premature real worker, and local paths in retained evidence. Each behavior change has a test or a deliberate fault.

Source: docs/SOURCE_REVIEW.md; docs/evidence/release-sensitivity.json

Stage-specific recovery wording

Failure explanations are tied to the stage that failed: a loading timeout never uses an image-analysis explanation, and Clear does not relabel a failed worker as a startup failure. A truthful unavailable state remains until retry succeeds. Controlled reproductions check the visible explanation, absence of old results and subsequent recovery (tests/test_recovery.py::test_model_load_timeout_clear_and_retry_describe_detector_availability, tests/test_recovery.py::test_inference_timeout_clear_and_retry_describe_detector_availability, tests/test_recovery.py::test_worker_crash_during_inference_is_not_described_as_startup_failure).

Source: docs/SOURCE_REVIEW.md; tests/test_recovery.py

Disposable-copy experiments validate the assertions. Besides the five core-rule faults in Verification and release assessment, eleven deliberate faults account for 17 of the 28 intended failures: blank visible names, blank visible scores, displaced boxes, removed announcement settings, a fixed load deadline, an extended initialization deadline, accepted blank labels, scores scaled by 0.95, scores forced to 1.0, a completed transfer left under the stall window and a cleared hang reported as an analysis failure. The record retains each source change and its intended failing assertion. A deliberate source fault demonstrates sensitivity to that specific mistake; it is not a production incident or a broad mutation-testing score.

Source: docs/evidence/release-sensitivity.json

Back to contents

Verification and release assessment

The recorded full suite passed: 102 passed, with 0 failures, errors or skips (98 controlled and 4 real-model cases), in 774.316 seconds of JUnit suite time. The run's commands, environment, source hashes and selected evidence are recorded in release-verification.json. The base commit is be34c5e; the recorded file hashes identify the tested files. Records of earlier commits overlap and are never added to this result.

Source: docs/evidence/release-verification.json; docs/evidence/release-full-suite.xml

Table 14 Execution records

Check	Observed result	Evidence and limit
Full suite	102 passed; 0 failures/errors/skips; 774.316 s JUnit	98 controlled and 4 real-model cases; release-full-suite.xml
Deliberate faults	30-case clean control passed; 16 faults, 28 intended failures; 1 equivalent fault passed as expected	release-sensitivity.json; faults planted in disposable copies
Rendered review	12 captures; 320/390/768/1440 px layouts fit	release-rendered-review.json
Static package	23 entries verified; 34,572,913 bytes in ZIP	release-package.json; prepared locally, not deployed
Slow-transfer probe	2.5 Mbps: ready at 140.6 s; fixed-deadline source failed at 121 s	branch-review-slow-transfer.json; loopback pacing, one run each; the fixed-deadline run used 17f2b9a, and the corrected package's loading code is identical to the release source

A fresh clone of the repository, followed through the README setup, usage example and test command, is recorded in docs/evidence/fresh-clone-run.json.

Earlier recorded runs

Each record covers the source it names, keyed by commit hash. Counts overlap and are never added to the result above. Run dates stay inside the evidence files; docs/VERIFICATION.md lists every record.

Table 15 Earlier recorded runs

Source commit	Recorded result	Evidence
764fb9d	102 passed; 780.398 s JUnit. Sensitivity: 13-case control, 11 faults, 17 intended failures	branch-review-full-suite.xml; branch-review-sensitivity.json
f1a19a9	98 passed; 783.414 s JUnit	loading-followup-full-suite.xml
17f2b9a	93 passed; 648.509 s JUnit	followup-full-suite.xml
74d108f	72 passed; 531.363 s JUnit	review-full-suite.xml
3ac9422	59 passed; 441.33 s runner	public-prep-full-suite.xml
781d9b7	48 passed; 418.62 s runner	extension-full-suite.xml
615f5c0 application files except dist/style.css	Repeated real-model use: 320 cycles in 3615.927 s; no observed failure	stability-summary.json; earlier source, not requalified; no flake-rate or memory-leak claim

The before/after recovery comparison ran the same three controlled wording checks against app.js from 74d108f and from 764fb9d, whose app.js is identical to the release source: three expected pre-Clear failures in 26.952 seconds, followed by three passes in 26.040 seconds. Only the passing checks on the fixed source continue through Clear, unavailable feedback, Retry and a valid new image. The baseline Clear behavior is not independently reached by those failed checks.

Source: docs/evidence/branch-review-recovery-comparison.json

The recorded environment is Windows 11, Python 3.12.10, pytest 9.0.3, Selenium 4.40.0, Chrome 156.0.8078.80 and ChromeDriver 156.0.8078.80. This was one Windows laptop with headless Chrome and prepared local assets. Workstation load was not controlled, so timing is an observation rather than a performance promise.

Source: docs/evidence/release-verification.json

Source and evidence hashes bind observations to inspectable bytes. Selected JUnit copies remove the machine hostname; the raw hashes and redaction are retained in the manifest. Scripts for sensitivity, before/after recovery comparison, rendered capture and verification recording are under tools.

The three real-photo box references are earlier pinned-model outputs, not independent annotations. Scores remain model scores, duplicate predictions are still possible and the filter only changes the displayed set. Malformed-response and timeout cases use deliberate controlled failures. The slow-transfer probe paced one response on loopback; it is not an Internet speed measurement. Announcement checks establish configured DOM behavior, not actual screen-reader output.

Core rules broken on purpose

tools/check_review_sensitivity.py copies the application into disposable folders and plants one deliberate fault in each copy; the checkout is never changed. In release-sensitivity.json a 30-case clean control passes, then each of 16 deliberate faults fails its intended tests, 28 intended failures in total. Five of the faults each break one core rule the question depends on.

Source: tools/check_review_sensitivity.py; docs/evidence/release-sensitivity.json

Table 16 Core rules broken on purpose

Core rule	Deliberate change	Test that failed
A file of exactly 10 MiB is accepted	file.size > MAX_BYTES becomes >=	tests/test_ui.py:: test_file_size_boundary[10485760-complete] (the 10 MiB + 1 byte case still passes)
The filter keeps a score equal to the minimum	score >= minimum becomes >	3 of 7 in tests/test_filter.py: test_exact_minimum_filters_underlying_scores_and_restores_without_inference test_filter_preserves_result_identity_color_and_surviving_selection test_hidden_predictions_can_be_restored_and_differ_from_no_model_predictions
An old image's result cannot repaint a newer image or a cleared workspace	the guard data.id === revision && currentImage?.id === revision becomes true	tests/test_ui.py:: test_replacing_a_processing_image_discards_the_old_result[slow-car.png] tests/test_ui.py:: test_clearing_a_processing_image_prevents_late_repaint
An outdated export cannot download	the check after PNG encoding (if (token !== exportRevision || snapshot.id !== revision) return;) removed	all 3 cases of tests/test_export.py:: test_changed_result_cancels_a_png_still_being_encoded
A late sample cannot replace a newer image or undo Clear	all three defences removed: the fetch abort in cancelSampleLoad, the check after the download, and the reserved revision passed to selectFile	both cases of tests/test_sample.py:: test_late_sample_response_cannot_override_a_later_user_action

One more fault is recorded as an equivalent fault: a change to the source that alters no observable behavior. Removing only the check after the sample download changes nothing observable, because the fetch abort and the reserved revision still stop the late sample, so its 2 cases are expected to pass, and they do. That is why the late-sample fault removes all three defences.

Conclusion: every core rule the question depends on has at least one test that fails when the rule is broken. The other 11 faults account for the remaining 17 intended failures (see Stage-specific recovery wording in Failure investigation). These results show that the selected checks detect the selected faults; they are not a mutation score or a test-completeness proof.

Release decision

The recorded result is 102 passed, 0 failures/errors/skips; 774.316 seconds of JUnit suite time (98 controlled and 4 real-model cases). A paced 2.5 Mbps model transfer that the fixed-deadline source could not complete reaches ready. Together with the targeted failure/recovery checks, deliberate-fault sensitivity and rendered review, it supports the local learning and demonstration workflow defined in docs/DESIGN.md. This document and the static ZIP are local deliverables. No website deployment or public URL is included.

Other browsers/devices, Safari/iOS, hosted CI, public deployment, a general accuracy dataset, GPU/NPU execution, semiconductor inspection and production scale remain outside the evidence. The one-hour repeated-use run (stability-summary.json) used an earlier source and does not qualify the tested one. Three known risks are not tested: repeated 45-second stalls, because Try again restarts the model download from the beginning; other browsers and mobile; and large real phone photos with EXIF rotation, because only synthetic images and small public photos are used. Repository content also cannot prove the owner's personal understanding or unaided authorship.

Back to contents

Running and maintaining the project

Use the application

Start the local server

```
.\.venv\Scripts\python.exe serve.py
```

Open http://127.0.0.1:8765 in Chrome and keep the terminal running. Ctrl+C stops the server. If that port is already in use, choose another with serve.py --port 8766 and open the corresponding local address.

For a short demonstration, use Try sample, wait for the real predictions, select a result, raise and lower the minimum score, then save the visible annotations. Open the resulting PNG separately and compare it with the current view. Finally choose a different photo or Clear.

Run checks

Execute the suite

```
.\.venv\Scripts\python.exe -m pytest
```

The tests own their temporary servers and headless browsers. They should clean up those resources when finished. A manually opened app can remain available in another terminal; do not confuse its state with the isolated test browser.

Table 17 Where to look

Location	Purpose
dist	The application and prepared local runtime/model/sample assets
tests	Python acceptance checks, server checks and browser fixtures
docs/DESIGN.md	The question, non-goals, acceptance examples, ranked risks and design decisions

Location	Purpose
docs/VERIFICATION.md and docs/evidence	Recorded results and selected evidence
docs/SOURCE_REVIEW.md	Defects and corrections, each in five parts
tools and asset-lock.json	Preparation helpers and exact asset sources/hashes
docs/system-design/source.json	Authoritative prose, tables and captions for this companion
docs/system-design/diagrams	Editable diagrams and their SVG/PNG exports
.venv, .local and artifacts	Generated environments and local working or raw-evidence files

Make a bounded change

Change one behavior with its acceptance criterion and relevant tests. Run the focused checks while developing, then the applicable wider suite and UI review before committing. A test that simply repeats the implementation's calculation can reproduce the same defect, so expected outcomes should follow the requirement or an independent example.

Keep the app's image identity, visible predictions and exported output consistent when changing result behavior. Keep real inference separate from controlled fixtures. Preserve existing attribution and the recorded asset sources when updating dependencies.

Update source.json and the relevant diagram source when the implementation changes, then rebuild the Word document and its matching PDF. The contents use internal hyperlinks and page numbers verified against the rendered document. Recheck those targets and every final page after changes. Editing a generated Word or PDF copy alone does not update the authoritative source.

AI assistance and honest demonstration

AI tools assisted with implementation, test authoring and documentation. The app adapts an upstream object-detection example and uses an existing pretrained model. It does not claim that this project trained the detector or that every automated check was independently written by the owner.

A demonstration can honestly describe the application workflow, the role of each technology, one acceptance criterion and its test, an investigated failure, and the scope of the release decision. The owner's understanding should be demonstrated through explanations and actions actually performed, without inventing prior experience.

The public package has an exact allowlist: application files, normal-mode configuration, six pinned runtime/model assets, the public coffee sample and license/provenance texts. Tests, documents, environments and Git history are excluded. An ordinary static-server integration test verifies the package locally. Source publication and website deployment are outside this package.

Source: tools/build_public_demo.py; tests/test_public_package.py; docs/PUBLIC_DEMO.md

Interview story (draft)

This is a draft. The owner revises it after studying the project, so that every sentence describes something the owner understands and can explain. It uses the Situation, Task, Action, Result shape.

Situation: Someone trying a browser object detector on their own photos can be misled: switching photos quickly, a slow connection, or a broken model response can show results for the wrong photo or leave the page stuck.

Task: I set out to show, with evidence, whether a small detection app could prevent those failures.

Action: I defined the scope and the question, and decided which review findings to fix. AI coding agents implemented the app and most of the tests, and I directed independent reviews of their work. One review throttled the model download to 2.5 Mbps and found that loading failed at 86% every time. I chose to bound loading by stalls instead of total time. I also had five core rules deliberately broken in throwaway copies, to prove the tests would catch them.

Result: The same 2.5 Mbps download now completes, ready at 140.6 s instead of failing at 121 s. The suite has 102 tests, and every deliberately broken core rule was caught. The lesson: ask what a timeout really protects against. Here it was a dead connection, not a slow one.

Back to contents

Terms and source references

Table 18 Terms in this application

Term	Meaning
Inference	Running a trained model on an input to produce predictions
Pretrained model	A model whose learned parameters were produced before this application used it
Model score	The number the model assigns to a prediction; not a measured accuracy percentage
Threshold	A selected minimum used to include or exclude predictions
Bounding box	A rectangle describing the predicted location of an object
WebAssembly	A browser execution format used by this runtime to run model operations on the CPU
Quantization	Using a lower-precision model representation; this application selects a prepared quantized model instead of training one
Web Worker	A browser execution context separate from the page's main thread
Fixture	Reusable test setup, inputs or controlled conditions
Assertion	A check comparing an observation with an expected rule
Regression	Previously correct behavior becoming incorrect after a change
Deliberate fault	A small change planted in a disposable copy to check that a test fails when a rule breaks
Equivalent fault	A deliberate change that alters no observable behavior, so no test is expected to fail
Stale response	A result belonging to an earlier image or operation
Acceptance criterion	A concrete observable condition the implemented feature must satisfy
Release decision	A conclusion about readiness for a stated use, based on evidence and known limits

Application behavior is established by the documented source and its retained execution records. Upstream documentation explains library concepts and APIs; it is not evidence that this application passed a check.

Table 19 Source index

Subject	Source
Question, scope, acceptance examples, risks and decisions	docs/DESIGN.md
User workflow and commands	README.md and docs/LEARNING.md
Application state and rendering	dist/app.js, dist/png-export.js, dist/index.html and dist/style.css
Actual model execution	dist/worker.js and asset-lock.json
Test setup and behavior	tests/conftest.py; UI, recovery, inference, startup, loading, filter, sample, export, server and public-package tests
Verification records	docs/VERIFICATION.md; release-verification.json; release-sensitivity.json
Defects and corrections	docs/SOURCE_REVIEW.md
Reproducible evidence	tools/check_review_sensitivity.py, check_recovery_comparison.py, capture_review.py and record_verification.py
Public sample provenance	docs/IMAGE_SOURCES.md
Upstream example and attribution	NOTICE.md and baseline/source.json
Transformers.js pipeline documentation	Transformers.js 2.17.2 pipeline API
Selenium documentation	Selenium browser automation
pytest documentation	pytest test runner

Back to contents